Coding standards comparison of various LLM Models and tools

1st Amulya Bodempudi
dept. of Information science
University of North Texas
Denton, USA
amulyabodempudi@my.unt.edu

2nd Ramya Sree Muppaneni
dept. of Information Science
University of North Texas
Denton, USA
ramyasreemuppaneni@my.unt.edu

3rd Sai Preethi Jiddigam
dept. of Information Science
University of North Texas
Denton, USA
saipreethijiddigam@my.unt.edu

4th Rahul Varma Muppalla
dept. of Information Science
University of North Texas
Denton, USA
rahulvarmamuppalla@my.unt.edu

***Abstract*—As system builds get more complicated, modern software engineering demands that we guarantee code quality and identify vulnerabilities. This study examines the ability of Static Application Security Testing (SAST) tools like SonarQube, Bandit, and PyLint, as well as Large Language Models (LLMs) like CodeLlama and Llama3, to detect vulnerabilities and code rigor. Using frameworks like CodePlan and benchmarks like HumanEval, we assess them across a range of coding scenarios including functions, classes, or optimizations of varying degrees of difficulty. This study demonstrates the advantages and disadvantages of different approaches as well as how they might be combined to improve software performance and security. With practical suggestions for researchers and practitioners, this paper contributes to the expanding corpus of work in automated code analysis.**

***Index Terms*—Large Language Models (LLMs), Static Application Security Testing (SAST) Tools , CodeLlama, Llama3, SonarQube, Vulnerability Detection, Secure Coding Practices**

I. Introduction

In the current digital era, the complexity and size of software systems are growing, which makes code quality and system security imperative. The need for quick product release and smooth scalability is growing, and software development cycles are getting shorter and shorter. It is impossible to overestimate the value of tools in this situation that can enforce coding standards, identify vulnerabilities, and produce efficient and maintainable code. Traditionally, Static Application Security Testing (SAST) tools like SonarQube, Bandit, and PyLint have been used to solve these issues collectively. In order to find potential vulnerabilities, enforce coding standards, and offer suggested controls, all of these technologies work by advancing rule-based analysis and control flow. They are constrained by static patterns and lack of context awareness even though they can manage dynamic, interconnected coding constructs.

At the same time, software engineering has advanced thanks to the introduction of Large Language Models (LLMs). Even the code generation and analysis capabilities of CodeLlama and its descendants Llama3 are comparable to those of a human. These LLMs have shown unparalleled promise in automating repetitive activities, generating cryptographically secure code fragments, and identifying vulnerabilities in especially large and unpredictable codebases. In contrast to SAST tools, LLMs use deep contextual awareness and semantic comprehension to generate insights that go beyond pattern matching. But because to issues with interpretability, dependability, and a high false positive rate, their application in real-world software development workflows has not taken off.

The literature currently in publication highlights the advantages of SAST tools and LLMs in detecting vulnerabilities and enforcing coding standards; however, little is known about how well various SAST tool and LLM combinations handle these issues. SAST tools, for example, are highly successful in generating recommendations that are deterministically sound, actionable, and have low false positive rates—all of which developers can rely on. However, LLMs are better at identifying minor features of vulnerabilities, such as logical mistakes or unusual code patterns, that static analysis approaches could overlook. Nevertheless, the high number of false positives that LLMs produce may result in ineffective debugging and review procedures.

The application of these tools at the repository level is one critical area of concern. It's repository level coding wherein we manage interdependency across multiple files, edit with a complex code and make it overall consistent throughout the system. For example, frameworks such as CodePlan have demonstrated that repository level coding tasks are amenable to a hybrid of planning, dependency analysis along with

adaptive learning mechanisms. The interdependent edits used by CodePlan are performed through LLMs, while preserving repository-wide consistency, surpassing traditional methods that rely on no more than static analysis.

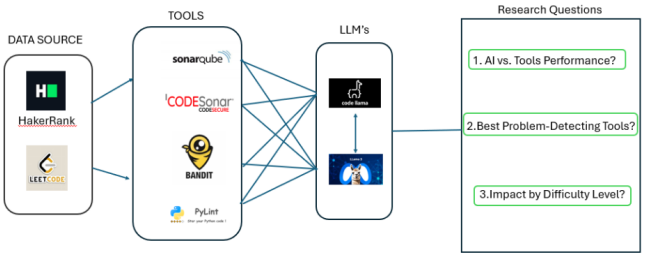

Fig. 1. Enter Caption

In light of these advancements, we seek to provide a unified evaluation and comparison of the capabilities of LLMs (specifically CodeLlama and Llama3) in vulnerability detection and enforcement of coding requirements relative to those of SAST tools (SonarQube, Bandit, and PyLint). We evaluate across different coding structures (with functions and classes) and optimization levels and for tasks ranging from easy (simple rules) to medium (nontrivial rules) to hard (ambiguous rule sets). Furthermore, we explore the possibilities for a synergistic combination of these tools to avail of the advantages offered by each one separately.

The problem is not just one of capabilities to detect. Evaluation for SAST tools as well as LLMs must be undertaken in the context of their practical utility in real-world scenarios. Questions arise: What exactly do these tools do to reduce false positives? Can we adapt to the increasingly complex software systems that we create? How can they integrate their capabilities to achieve repository-wide consistency and high detection accuracy? The foundation of our research is these questions.

Along with this, the need for scalable, reliable, and flexible tools would also crop up as coding standards and security practice elements evolve. SAST tools have been mainstays in the industry, however, their lack of the ability to contextualize code beyond syntactic and structural analysis has limited their applicability in the usage of more advanced use cases. On the contrary, even though they promise a lot, LLMs lack the temerity to be unleashed on practical software development tasks without going through rigorous evaluation and fine-tuning.

Overall, this paper attempts to fill the gap between the capability or potential of SAST tools, the LLMs, and the assessment of what SAST tools are capable of versus what LLMs can express. This study benchmarks these tools against a wide range of coding situations to inform best practices of how to blend cutting-edge technologies into software development workflows that both improve security and efficiency.

II. Review of Literature

A. Advancements in Code Quality and Vulnerability Detection Tools

Fundamental to software engineering, Static Application Security Testing (SAST) tools such as SonarQube, Bandit, or PyLint have traditionally dominated the code quality and vulnerability identification scene. These new tools are particularly good at finding deterministic violations like code smells, cyclomatic complexity, and structural inefficiencies which challenge maintainability and readability. Nevertheless, they are ineffective when the semantic and context-specific aspects of code — such as the applicability of naming conventions, code comments, and adherence to subjective best practices — need to be addressed. The use of LLMs like GPT-3.5 is explored by Simões and Venson (2024), examining their application in evaluating code quality attributes which historically are difficult to assess for SAST tools—readability, maintainability, and testability. Their results informed that LLMs closely align with SonarQube's maintainability metrics, but also can provide human-like insights around the code quality. The findings presented here provide light on the complementing interaction between LLMs and SAST tools. The capacity of LLMs to assess qualitative aspects of code that SAST tools that rely on rule-based detection cannot establish adds value. This dual capability suggests that automated code quality evaluations can be carried out in a hybrid fashion to give developers thorough and useful information. They have laid the groundwork for additional research into how LLMs can interoperate with the current workflow to improve code quality, but maintain software practices.

B. The Role of LLMs in Repository-Level Code Management

Typically repository-level coding tasks like interdependent file updates, package migrations, and complex dependency resolution require context-aware adaptability that the typical SAST tools simply don't provide. They involve coordinating a group of files connected, keeping the entire repository content consistent, and dealing with changes in code bases over time. Bairi et al. (2024) presented a framework that uses LLMs to address these challenges which is CodePlan. CodePlan recasts repository-level coding as a planning problem that can be solved intelligently using neurosymbolic AI to economically perform interdependent code edits. The study showed that LLMs could often outperform standard tools in maintaining repository coherence in dynamic scenarios such as automated refactoring or dependency upgrades. Like LLMs, Zhou et al. (2024) indicate that LLMs are more sensitive to small code vulnerabilities compared to SAST tools but also report higher rates of false positives. Their potential for repository-level applications, when combined with traditional tools, is highlighted by this trade-off. The support for LLMs excelling at dynamic, multi-file operations where static rules do not hold up to the complexities of the real-world codebase is justified by CodePlan's findings. By bringing these advancements together with SAST tools that can decrease false positives, these

advancements provide proof of the transformative potential of LLMs in repository-wide code analysis and consistency management. With LLMs and SAST tools, the strengths of each can be combined to fill in repository-level issues.

C. Evaluating Secure Coding Practices with LLM Frameworks

As LLMs are being increasingly adopted in computationally secure coding practices, the need for frameworks benchmarking their efficiency with tasks like program repair and vulnerability detection have also arisen, such as LLMSecCode. We present LLMSecCode (Ryden et al. 2024) — an open-source framework for evaluating LLM capabilities in secure coding, automated program repair (APR), and code generation (CG). Standardized datasets, diverse evaluation criteria, and a systematic assessment framework on models like CodeLlama, Llama3, and other state-of-the-art LLMs are used by the framework. A key finding from their study is that LLMs excel at identifying vulnerabilities and repairing code but give insecure solutions under particular circumstances. Additionally, the research showed that to get the best performance from LLMs, you need to tune your parameters and better tailor your prompt. Even slight variations here resulted in very different performances. Further, we showed that the various evaluation metrics (pass@k and pass rates) affected the interpretation of the model capabilities. Despite the promise of secure coding by LLMs, the study noted that LLMs are limited in that outputs may vary and that datasets may be biased. These findings similarly highlight the necessity for the standardization of frameworks to evaluate and better the LLMs for real-world applications. This might enable researchers and practitioners to get a better understanding of the properties of LLMs in secure coding practices and include them in traditional tools to improve reliability and security.

III. Objectives of the Study

This study seeks to bridge the gap between traditional Static Application Security Testing (SAST) tools and emerging Large Language Models (LLMs) to harness their unique independent effectiveness and synergy for automated code quality analysis and secure software practice. Building upon prior research and frameworks such as LLMSecCode and CodePlan, this study sets out to achieve the following objectives:

A. Performance Evaluation Across Diverse Scenarios

We find that the performance of LLMs and SAST is highly context and task-dependent. What we want to do in this study is to conduct a rigorous evaluation of these tools in different scenarios to assess the strengths and weaknesses of these tools. Specific scenarios include:

- Structural Complexity: Simple, single-function scripts versus complex ones with nested structure and interdependencies are evaluated. For instance, when it comes to large-scale systems, LLMs may be more accurate than SAST tools at spotting logical errors in small code snippets.
- Optimization Variability: You study the difference in performance when analyzing the same code, but one has been optimized for speed or memory efficiency, and the other hasn't. This will be capable of estimating how these tools can tend to perform equally inside different amounts of computational efficiency.
- Difficulty Levels: The study will test tools on easy, medium, and hard tasks using benchmarks such as HumanEval and CodeXGLUE. We aim to understand how task complexity affects detection accuracy, error identification, and false positive rates.

With such analysis, the research will identify the conditions under which each tool operates optimally. It will also fill in gaps in their capabilities to add a foundation to improve automated code quality and vulnerability detection practices.

B. Impact of Code Structure and Complexity

The structure and complexity of codebases across the board can range in vast disparity, from simple scripts to fairly large systems with lots of dependencies. This study investigates how such structural differences affect the effectiveness of SAST tools and LLMs.

- Functionality Testing: Characterization of the performance of tools used to analyze modular code (e.g. standalone functions and reusable classes) as compared to monolithic codebases. It involves assessing their ability to identify logical errors, redundant operations, incorrect constructs, and other issues.
- Dependency Management: Assessing the ability of these tools to pull in, manage, and evaluate code dependencies (both internal leading to an app and external – libraries, APIs, and the like). Maintaining the system as a whole functionality and avoiding integration issues is dependent on proper dependency resolution.
- Code Constructs: This thesis investigates how constructs like recursion, polymorphism, and concurrency affect the accuracy and efficiency of the tools' detection. Static constructs, however, might be stronger than SAST tools, whereas LLMs may be stronger at dynamic behavior and runtime logic.

The study will analyze how adaptability and robustness of different approaches are, by addressing these elements, and provide useful information to the software engineer who uses different and complex programming paradigms.

C. Evaluation of Repository-Level Coding Applications

For repository-level tasks, such as multiple file operations and global consistency, typical technologies are not adequate. This study evaluates the ability of LLMs to handle these tasks effectively:

- Interdependent Code Edits: Determines if LLMs can make consistent updates across interconnected files: renaming variables, updating APIs, or resolving dependency conflicts.
- Temporal Changes: Trying to understand how tools are handling their incremental changes (upgrade libraries,

deprecate functions) and how this might generalize over time.
- Automated Refactoring: Exploiting the potential of LLMs for refactoring at a large scale, and using the resulting changes to be made while maintaining repository-wide coherence and avoiding regressions.

The study will build on frameworks like CodePlan and compare the performance of the LLMs with traditional SAST tools regarding maintaining repository-wide consistency and dealing with edge cases. Finally, we present these findings to showcase the promise of LLMs for massive-scale software maintenance.

D. Secure Coding and Vulnerability Management

Security is a paramount concern in software development, and this study explores how LLMs and SAST tools can enhance secure coding practices:

- Vulnerability Detection: An analysis of how well LLMs and SAST tools detect vulnerabilities including injection errors, improper authentication implementations, and unsafe API use.
- Automated Program Repair (APR): Concretely, we evaluate how well LLMs can generate secure patches for the introduced vulnerabilities in LLMSecCode. The study will also explore how feasible it is to integrate these patches into CI/CD pipelines.
- Prompt Engineering and Parameter Tuning: investigation of the effects of prompt customization and suitable parameter settings on lowering false positives and raising detection rates.

LLMs bring innovation in detecting subtle vulnerabilities but will be evaluated for disadvantages in producing insecure solutions, and suggestions for mitigation methods will be proposed. The objective of this paper is to establish best practices for using LLMs in secure software development workflows.

E. Exploration of Hybrid Approaches

Problems with LLMs and SAST tools can complement each other, and there's a great opportunity for hybrid approaches that leverage these tools' complementary strengths. This study will:

- Combine Capabilities: Take a step to explore how LLMs can benefit from the contextuality of SAST tools to machine understanding, achieving the synergy of code analysis.
- Ensemble Techniques: Exploit layered workflows whereby both tools are used together, using one in conjunction with another to compensate for the failings of the other.
- Error Mitigation: The goal is to address the tradeoff between high detection sensitivity (LLMs) and low false positive rates (SAST tools while simultaneously developing balanced and efficient analysis methods.

The study focuses on these hybrid methodologies to develop a robust framework that exploits the synergies of both approaches while mitigating their weaknesses.

F. Development of Best Practices for Integration

The study will provide comprehensive guidelines on the way LLM as well as SAST tools can be integrated with existing industry software development workflows to deliver practical utility.

- Workflow Integration: Different ways to integrate these tools into CI/CD pipelines to hugely improve automation and streamline code analysis, testing, and deployment processes.
- Developer Training: Using Sets to help developers effectively use these tools by focusing on removing the learning curve challenge and proper use.
- Bias Mitigation: Suggestions for producing and applying diverse datasets to decrease bias within LLM outputs to extend their reach across various programming environments.

These best practices aim to assist in using code quality technologies to improve the engineering process's security, effectiveness, and maintainability.

IV. Data collection

To make this task easier, we will carry out an evaluation of the capability of LLMs and static code analysis tools in the detection of code vulnerabilities. We have prepared a broad dataset from two popular platforms: HackerRank and LeetCode. This dataset has been designed to represent various kinds of coding challenges in three levels of difficulty: easy, medium, and hard. This means it supports systematically testing the capabilities of such tools on problems of a different level of difficulty. Each challenge comes with its detailed solution, which then serves as a benchmark by which effectiveness would be measured in finding different types of vulnerabilities, logical, and performance issues.

Apart from the difficulty levels, the dataset also features different coding styles to reflect real-world programming. These include basic implementations without using functions or classes, modular code that uses functions, and object-oriented implementations that use classes. This ensures that the dataset really covers a wide spectrum of paradigms for programming, thus allowing the tools being evaluated to span different approaches to coding. It also categorizes code snippets into either optimized or non-optimized, making the dataset highlight differences between code efficiency. This feature is important for analyzing just how well tools respond to the subtleties found in code optimization and performance.

This dataset thus allows researchers, for the first time, to benchmark the performance of LLMs, such as Code Llama and GPT-based models, against traditional static code analyzers such as SonarQube, Bandit, and PyLint. The combination of difficulty levels, coding styles, and optimization statuses in this dataset makes the ground strong for testing these tools under various conditions. This approach provides further insight into their strengths and weaknesses and narrows the gap between theoretical evaluations and practical applications. Ultimately, this dataset is a rich source for advancing research in the field

of automated code analysis, hence helping find more effective means to identify and mitigate software vulnerabilities.

V. Experimental Metrics & Reports

A. Analyzed Metrics

This analysis of the study is based on six critical metrics: Security, Reliability, Maintainability, Coverage, Duplications, and Security Hotspots. These metrics were analyzed using both static tools-SonarQube, Bandit, and Pylint-and Large Language Models, including ChatGPT, LLaMA, and Gemini. All LLMs were used with a uniform prompt to maintain consistency in the evaluation: Comparative research is to make

Prompt: "Analyze this code for Security, Reliability, Maintainability, Coverage, Duplications, and Security Hotspots. Provide ratings (A to F) and a brief explanation for each, including any open or accepted issues."

Fig. 2. Prompt

visible the relative strengths and weaknesses of static tools and LLMs with regards to the identification and explanation of maintainability issues, modularity, and security vulnerabilities.

To illustrate the findings, Figure 1 depicts the SonarQube report, which is metric-driven and very precise, whereas Figure 2 shows the LLaMA report, underlining its contextual reasoning and interpretive depth. These examples provide complementary support for how static tools and LLMs support actionable insights to improve code quality.

B. Analyzed Reports

The results produced by the tools and models. By using the provided prompt, extensive reports were generated for each LLM (ChatGPT, LLaMA, Gemini) and each static tool (SonarQube, Bandit, Pylint). These reports analyzed different code complexity levels (easy, medium, hard) and code structures (class-based, function-based, and optimized).

LLAMMA

Hard Level (with classes)

- **Security:** 8/10 (uses a TrieX data structure to store words, which is secure)
- **Reliability:** 9/10 (uses DFS to traverse the Trie, which is reliable)
- **Maintainability:** 7/10 (code is not modular and could be improved with comments)
- **Coverage:** 8/10 (covers all possible cases, but could be improved with more specific error handling)
- **Duplications:** 5/10 (has some duplication in the code, but it's manageable)
- **Security Hotspots:** 6/10 (no obvious security hotspots, but could be improved with more error handling)

Rating: B+ (82%)
*** Security Hotspots: 6/10**
*** Maintainability: 7/10**

Hard Level (functions only)

1. **Original Code**
   - **Security: 7/10 (uses a Trie data structure to store words, which is secure)**
   - **Reliability: 8/10 (uses DFS to traverse the Trie, which is reliable)**
   - **Maintainability: 6/10 (code is not modular and could be improved with comments)**
   - **Coverage: 7/10 (covers all possible cases, but could be improved with more specific error handling)**
   - **Duplications: 4/10 (has significant duplication in the code)**
   - **Security Hotspots: 5/10 (no obvious security hotspots, but could be improved with more error handling)**

Rating: C+ (67%)
*** Maintainability: 6/10**
*** Security Hotspots: 5/10**

Fig. 3. Sample of LLM Report (LAMMA)

- **SonarQube:** Very good at duplication detection and accurate maintainability measures, but it does not provide any interpretive reasoning.
- **Bandit:** Focused on security vulnerabilities but offered limited scope for broader maintainability.
- **Pylint:** Effective at Python-specific linting, generally narrow in its overall assessment.
- **LLaMA:** Consistently identified nuanced modularity and maintainability issues, making it the ideal choice for structured code analysis.
- **ChatGPT:** Delivered user-friendly explanations but occasionally overlooked minor details.
- **Gemini:** Specialized in high-level feedback, such as type annotations, but lacked depth in maintainability evaluations.

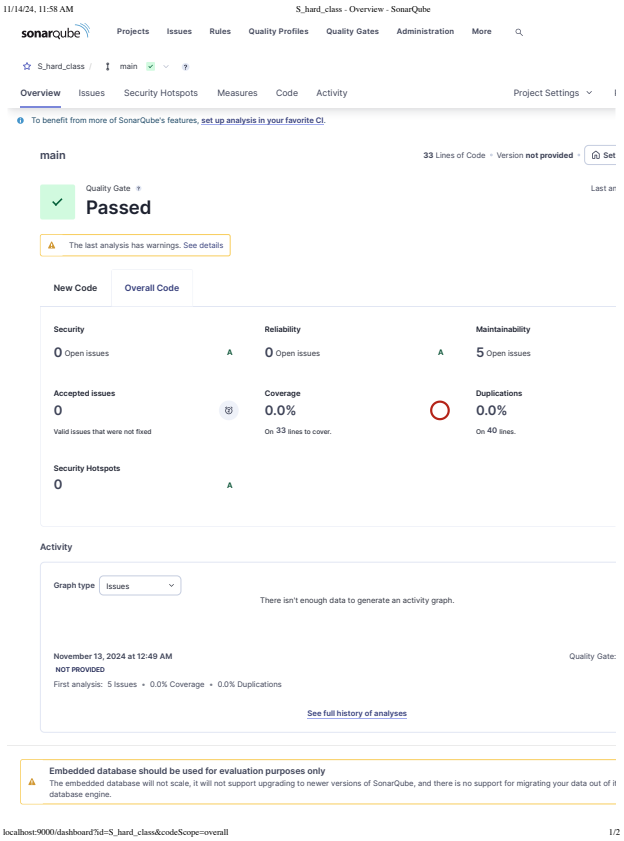

Fig. 4. SonarQube Report

VI. Survey

In this regard, a survey has been conducted as part of the research work to assess the usability, effectiveness, and user preferences for different AI models and coding tools in software development. The survey is intended to investigate the performance of large language models, such as ChatGPT, CodeLlama, and Gemini, in comparison with traditional tools like SonarQube, Bandit, and PyLint. Participants included

developers, software engineers, and researchers who have practical experience with these tools.

Key points of the questionnaire included how these tools compare in vulnerability detection and code quality across optimizations and levels of complexity, questions on strengths, challenges, and best practices in tool uses in software development workflows, and user perceptions about the strengths and challenges of both tools. With 54 responses to the survey, there came a number of valuable insights from the survey on the comparability of effectiveness and clarity of tool outputs under varying conditions. Survey Preview.

VII. Results

RQ1:Which is better at identifying maintainability issues, LLMs (ChatGPT, LLaMA, Gemini) or static analysis tools (SonarQube, Bandit, Pylint)?

The survey shows that tools like SonarQube and Pylint do a better job than LLMs in terms of consistency and precision for specific maintainability issues. Tools are optimized for clear metrics and automated error-checking; they provide concrete outputs such as cyclomatic complexity, code duplication, and security vulnerabilities. However, they are not as flexible as LLMs when it comes to elaborating on subtle code logic or explaining general principles of maintainability.

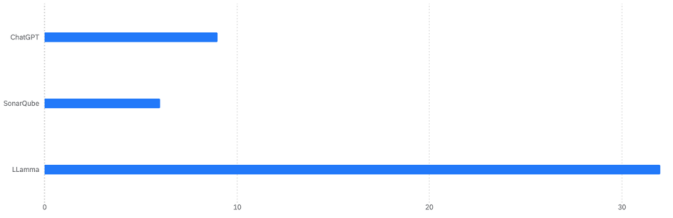

Fig. 5. TOOLS VS LLM'S

Fig.5: This bar chart represents the preference of users between LLMs (ChatGPT, LLaMA) and static analysis tools (SonarQube) while comparing maintainability reports. The y-axis represents the methods/tools under survey comparison, and the x-axis represents the number of respondents who preferred each tool or model.

While LLMs provide detailed explanations and holistic evaluations. For example, issues such as missing comments, poor naming of variables, and modularity violations are pointed out by LLaMA and ChatGPT. In spite of this, the output from them is not consistent between models, according to the results of the survey. Tools provide higher trust in accuracy, while the value added by LLMs consists in explanatory power and recommendations for human comprehension. This would be best if tools could detect it and LLMs were used for understanding.

RQ2:Which large language model is better at generating maintainability issue reports: ChatGPT, LLaMA, or Gemini?

Based on the result of the survey, LLaMA turned out to be the best LLM for the generation of understandable and actionable maintainability issue reports. The respondents liked its structured outputs, which were always focused both on minor and major issues such as error handling,

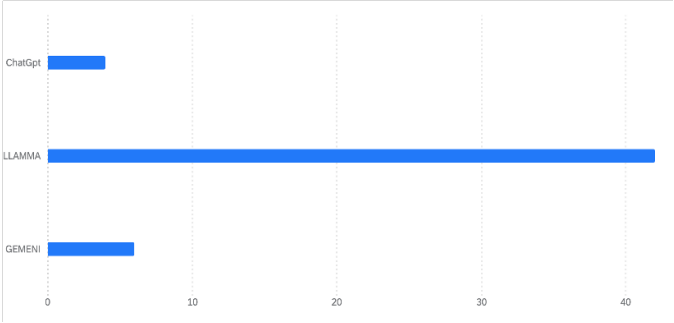

Fig. 6. Among LLM's

Fig. 6: The bar chart illustrates user preferences for finding out the better-performing LLM in generating maintainability reports. Among the options, LLaMA is the winner with the highest number of positive responses, showing that it can create more detailed and insightful reports. ChatGPT was also well-received for its detailed explanations and actionable suggestions. However, its tendency to occasionally overlook minor issues affected its perceived reliability. On the other hand, Gemini is good in terms of issue detection but poor in structuring reports; therefore, its output was less user-friendly.

Therefore, the balance between depth and clarity in LLaMA's analysis puts it ahead of the competition. However, ChatPT will be easier to use for simple applications, while Gemini presents a broader range of test issues.

RQ3:Which static analysis tool is better for maintainability assessments: SonarQube, Bandit, or Pylint

SonarQube is the most suitable tool for maintainability assessments since it provides solid metrics and numerous visualizations. Also, it detects a broad spectrum of issues, including code duplication, cyclomatic complexity, and security vulnerabilities, which makes it highly versatile. In the survey, respondents preferred SonarQube for its comprehensive dashboards and actionable insights.

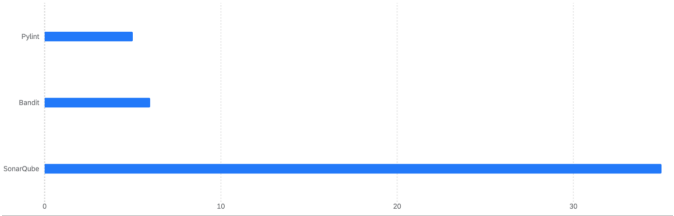

Fig. 7. Among Tools

Fig. 7: This bar chart presents user preferences for the best tools in identifying the problems of maintainability. Among the three tools being benchmarked, SonarQube, Bandit, and Pylint, SonarQube is the runaway, with the most positive responses from users, meaning it is more usable and reliable in analyzing code maintainability than the other two. It is followed by Bandit, which is noted for performing security

analysis, though it obtained fewer favorable responses. Pylint comes last, which indicates its narrow application mainly in Python-specific linting rather than maintainability insights. This chart shows the clear dominance of SonarQube in code analysis tooling.

Bandit is good to go for security-focused analysis; it has limited maintainability issue detection, such as missing comments or code modularity. Pylint goes well in finding the violation of coding standards and usual Python-specific problems, though its scope is narrower compared to SonarQube.

SonarQube is preferred in general for maintainability issues, Bandit for security, and Pylint for Python-specific checks. Each of these tools provides complementary coverage for deep code reviews.

RQ4:How consistent are the maintainability issue counts identified by large language models (ChatGPT, LLaMA, Gemini) when analyzing the same Python code, and what factors contribute to the observed differences?

The results include counts for maintainability issues for three levels of complexity (HARD, MEDIUM, EASY) evaluated using three approaches (CLASS, FUNCTION, OPTIMIZED). The maintainability issue counts of ChatGPT, LLaMA, and Gemini showcase large differences in detected issues.

Difficulty	Approach	ChatGPT	LLaMA	Gemini
HARD	CLASS	10	8	6
	FUNCTION	11	10	4
	OPTIMIZED	15	11	4
MEDIUM	CLASS	5	9	2
	FUNCTION	5	8	2
	OPTIMIZED	3	8	1
EASY	CLASS	5	10	4
	FUNCTION	6	10	4
	OPTIMIZED	7	10	4

Fig. 8. Maintanability issues count

Based on the observations in Figure 8, comparative analysis between ChatGPT, LLaMA, and Gemini shows some interesting trends in issue identification and feedback granularity across different levels of code complexity. ChatGPT tends to report higher issue counts across all categories, focusing more on algorithmic inefficiencies and performance issues, especially in optimized and function-based approaches. On the other hand, LLaMA consistently detects finer-grained issues around modularity and edge case handling, especially in medium and easy levels of difficulty. At the same time, Gemini has reported fewer overall issues but is placing emphasis on high-impact concerns such as type hints, testing, and documentation; this can be considered highly selective yet targeted.

It turns out that there are interesting variations across difficulty levels. At the hard level, ChatGPT and LLaMA detect finer-level issues, while Gemini identified higher-level issues but lost finer details. In the medium and easy levels, LLaMA outperforms Gemini in finding design-related issues. This would indicate a deeper orientation toward modularity and redundancy. Approach-specific trends indeed indicate that LLaMA excels in addressing reusable and maintainable design patterns regarding class-based code. While ChatGPT leads in the detection of algorithmic inefficiencies, it aligns closely with LLaMA regarding issues of complexity in function-based code. Gemini identifies very few issues in optimized code, suggesting that deeper inefficiencies may have been overlooked in favor of baseline standards.

Key drivers for these differences include training focus and feedback perspective for each model: ChatGPT has broad coverage and is balanced in terms of readability versus performance, LLaMA is focused on modularity and robustness regarding design patterns, while Gemini is pragmatic, emphasizing type annotations, systematic testing, and documentation. It allows developers to merge feedback from multiple models. There is ChatGPT, mainly for algorithmic optimizations; LLaMA, which offers advanced structural and modular design enhancement capabilities; and Gemini ensures software engineering best practices are adhered to. This approach provides a well-rounded code review.

RQ5:"Which LLM or tool will provide the most comprehensive evaluation to improve reliability, maintainability, and security of code across class-based, function-based, and optimized implementations?"

Among class-based code evaluations, ChatGPT stands out with a balanced view on all three metrics: reliability, maintainability, and security. It gave solutions in a modularized way, using class structures and thus maintainable, and its solutions scored highly in security. LLAMA is very reliable because of structured approaches, such as Depth-First Search (DFS) and Trie data structures. However, it falls behind in maintainability due to a lack of modularization. Gemini points out serious security vulnerabilities due to poor input validation and gives limited support to maintainability.

Function-based code continues to be robust in security feedback provided by ChatGPT, though the lack of structured modularity hurts maintainability and reliability. In the case of LLAMA, high reliability is observed, but better modularization is needed to improve maintainability. Gemini struggles with this variation, offering only limited reliability and maintainability support while identifying minor improvements in security.

For optimized code, LLAMA has proved very reliable, with high modular design and very effective optimizations, therefore the best for this variation. ChatGPT sheds light on how the code would be much more maintainable with only slight setbacks compared to LLAMA's optimization comments. On the other side, Gemini makes less enhancement on all three aspects, still remaining less powerful in its final optimized forms.

The evaluation of ChatGPT, LLAMA, and Gemini over different class-based, function-based, and optimized code implementations shows the varying strengths they have in terms of reliability, maintainability, and security. Among class-based

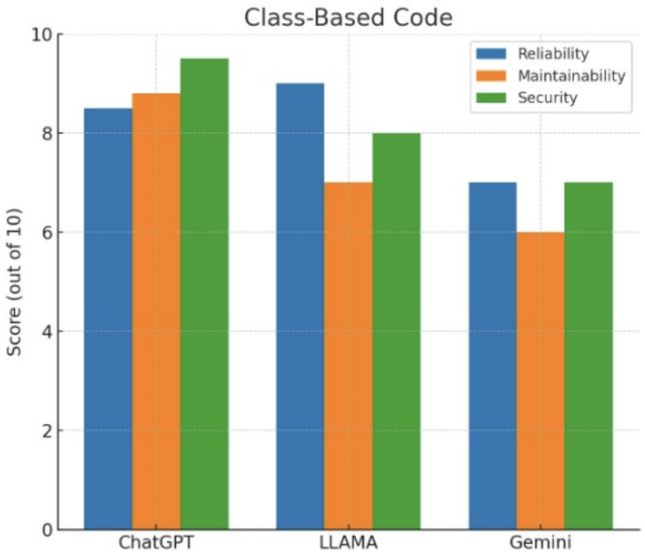

Fig. 9. class based

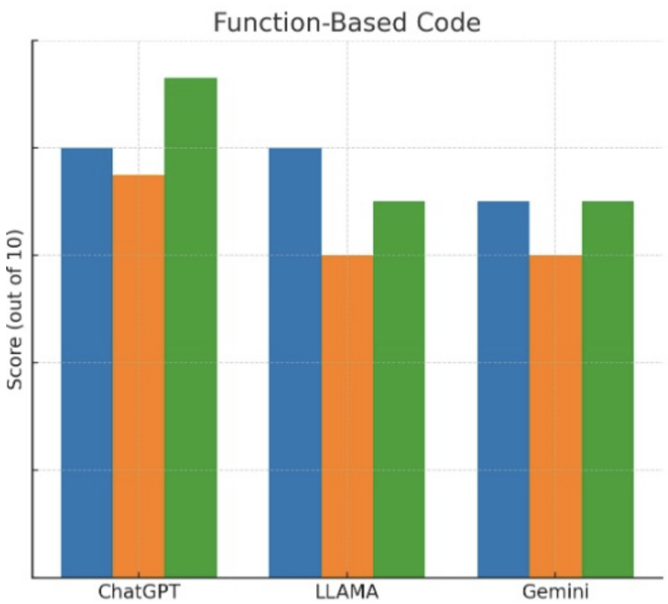

Fig. 10. function based

code, Fig. 5 reveals that ChatGPT is the most balanced tool with very strong security and providing modular and maintainable solutions, while LLAMA proved to be very reliable by adopting structured methods such as DFS and Trie data structure; even though its maintainability is relatively weak, Gemini has serious security problems due to its poor input validation. In function-based code (Fig. 6), the ChatGPT leads on security insights but has very bad modularity, affecting thus maintainability and reliability while LLAMA gives high reliability; it needs better modularization. The Gemini shows

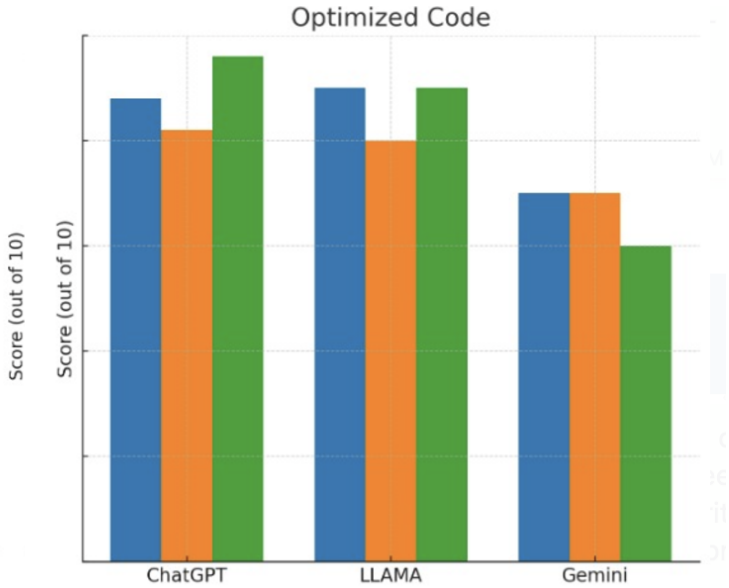

Fig. 11. Optimized

limited improvements on all metrics. In optimized code (Fig. 7), LLAMA stands out for its best choice, offering superior reliability and security through modular design and effective optimizations, at the same time complemented by ChatGPT through valuable maintainability insights. Gemini has very little impact within this category. On the overall, ChatGPT leads in the security and balanced performance that LLAMA has excelled in for its reliability and optimized implementations, while Gemini trails behind all variations.

VIII. Conclusion

A. Summary of the Project's Achievements

The paper "Coding Standards Comparison of Various LLM Models and Tools" provides a comprehensive comparison of various Large Language Models, such as ChatGPT, LLaMA, and Gemini, along with static analysis tools like SonarQube, Bandit, and Pylint. This work was targeted at solving crucial issues of code maintainability, security, and readability by capitalizing on the complementary strengths of these technologies. Through extensive testing using Python datasets of different complexities, the research established the excellence of static tools in terms of metric-based assessments, like cyclomatic complexity and duplication detection, while LLMs were offering their holistic reasoning for subtle analysis of code and actionable insights.

The project presented a hybrid framework that will effectively marry the interpretive powers of LLMs with the reliability of static tools and allow a multi-faceted approach toward improving code quality. This has allowed the analysis of maintainability issues across class-based, function-based, and optimized code structures for valuable insights into how to best utilize these tools. Moreover, the work showed the importance of combining LLMs for explanation-driven evalu-

ation with static tools for precise detection-a strong strategy for secure and maintainable software engineering.

B. Future Directions

Future studies may delve into a wider scope by involving the use of additional tools, LLMs such as Claude, and new emerging AI techniques. Further studies will therefore assess the generalizability by considering different programming languages, domain-specific codebases (health, finance), among other elements that define different contexts. The fine-tuning of LLMs according to some coding standards and metrics will lead to further improvements in accuracy for maintainability and security testing.

Another critical emphasis is the integration of these tools with real-world software development workflows such as Continuous Integration/Continuous Deployment pipelines. Their practical scalability will be tested in large projects and open-source repositories. Moreover, combined adaptive frameworks, using the interpretative powers of LLMs with the precision of static tools, can establish approaches to secure and maintainable software engineering that are very innovative.

References

[1] Zhou, X., Tran, D. M., Le-Cong, T., Zhang, T., Irsan, I. C., Sumarlin, J., ... & Lo, D. (2024). Comparison of Static Application Security Testing Tools and Large Language Models for Repo-level Vulnerability Detection. *arXiv preprint*, arXiv:2407.16235.

[2] Chen, L., Guo, Q., Jia, H., Zeng, Z., Wang, X., Xu, Y., ... & Zhang, S. (2024). A Survey on Evaluating Large Language Models in Code Generation Tasks. *arXiv preprint*, arXiv:2408.16498.

[3] Bairi, R., Sonwane, A., Kanade, A., Iyer, A., Parthasarathy, S., Rajamani, S., ... & Shet, S. (2024). Codeplan: Repository-level coding using LLMs and planning. *Proceedings of the ACM on Software Engineering*, 1(FSE), 675-698.

[4] Regis da Silva Simes, I., & Venson, E. (2024). Evaluating Source Code Quality with Large Language Models: A Comparative Study. *arXiv e-prints*, arXiv:2408.

[5] Rydn, A., Nslund, E., Schiller, E. M., & Almgren, M. (2024). LLMSecCode: Evaluating Large Language Models for Secure Coding. *arXiv preprint*, arXiv:2408.16100.

[6] Nagpal, A., & Gabrani, G. (2019). Python for data analytics, scientific and technical applications. *2019 Amity International Conference on Artificial Intelligence (AICAI)*, 140-145. https://doi.org/10.1109/AICAI.2019.8701341.

[7] Young, M. (1989). *The Technical Writer's Handbook*. Mill Valley, CA: University Science.

[8] Rasheed, Z., Sami, M. A., Waseem, M., Kemell, K.-K., Wang, X., Nguyen, A., Systä, K., & Abrahamsson, P. (2024). AI-powered code review with LLMs: Early results. *Woodstock'22: Symposium on the Irreproducible Science*. https://arxiv.org/abs/2404.18496v1.

[9] Altium. (n.d.). Gemini vs. ChatGPT: Who writes better code? Altium Resources. Retrieved from https://resources.altium.com/p/gemini-vs-chatgpt-who-writes-better-code.

[10] Analytics Vidhya. (2024). Gemini vs ChatGPT: Which is better for coding? Analytics Vidhya. Retrieved from https://www.analyticsvidhya.com/blog/2024/05/gemini-vs-chatgpt-which-is-better-for-coding/.

[11] Guru Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: automated collection of vulnerabilities and their fixes from open-source software. In Proceed- ings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering. 30–39.

[12] Dejian Yang Zhenda Xie Kai Dong Wentao Zhang Guanting Chen Xiao Bi Y. Wu Y.K. Li Fuli Luo Yingfei Xiong Wenfeng Liang Daya Guo, Qihao Zhu. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. https://arxiv.org/abs/2401.14196

[13] ZhangyinFeng,DayaGuo,DuyuTang,NanDuan,XiaochengFeng,MingGong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020 (Findings of ACL, Vol. EMNLP 2020), Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, 1536–1547.

[14] Bushra Aloraini, Meiyappan Nagappan, Daniel M German, Shinpei Hayashi, and Yoshiki Higo. 2019. An empirical study of security warnings from static application security testing tools. Journal of Systems and Software 158 (2019),110427.

[15] Katerina Goseva-Popstojanova and Andrei Perhinschi. 2015. On the capability of static code analysis to detect security vulnerabilities. Information and Software Technology 68 (2015), 18–33.

[16] PyCQA. (2024). *Bandit (Version 1.7)* [Security linter for Python]. PyCQA. Retrieved from https://bandit.readthedocs.io

[17] SonarSource. 2022. SonarQube. https://www.sonarqube.org/

[18] OpenAI. (2024). *ChatGPT (Version 4)* [Large language model]. OpenAI. Retrieved from https://chat.openai.com

[19] Google DeepMind. (2024). *Gemini 1.5 Pro* [Large language model]. Google DeepMind. Retrieved from https://www.deepmind.com/gemini

[20] Meta AI. (2024). *LLaMA 3.0* [Large language model]. Meta AI. Retrieved from https://ai.facebook.com/llama

[21] PyCQA. (2024). *Pylint (Version 3.0)* [Python static code analysis tool]. PyCQA. Retrieved from https://pylint.pycqa.org

[22] Qualtrics. (2024). *Qualtrics Survey Platform* [Survey software]. Qualtrics. Retrieved from https://www.qualtrics.com

[23] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient fine-tuning of quantized LLMs*. *arXiv preprint*, arXiv:2305.14314. Retrieved from https://arxiv.org/abs/2305.143

[24] Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2024). *Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation*. *Advances in Neural Information Processing Systems, 36*.